

YANTRA • UNITY 6000.3.15f1

คู่มือการใช้งาน YUKI Learning State Machine

แนวทางสร้าง State, Action, Condition และ Transition Table โดย
อ้างอิงตัวอย่าง Rin ใน Scene **TestStateMachine**

ตัวอย่างอ้างอิง: **Assets/YukiLearningStateMachine/Scene/TestStateMachine.unity**
ชุดข้อมูลตัวอย่าง: **Assets/YukiLearningStateMachine/ScriptableObject/Rin**

คำเตือนสำคัญ
ห้ามแก้ไขโค้ดทุกไฟล์ในโฟลเดอร์ **StateMachine** เด็ด
ขาด

1. ขอบเขตและกฎสำคัญ

State Machine ชุดนี้แยก “ระบบแกนกลาง” ออกจาก “พฤติกรรมเฉพาะตัวละคร” ผู้ใช้งานควรสร้าง Action และ Condition ของตนเอง แล้วนำมาเชื่อมกันผ่าน ScriptableObject ใน Inspector

ห้ามแก้ไข เพิ่มโค้ด หรือลบโค้ดภายใน `Assets/YukiLearningStateMachine/Script/StateMachine` เด็ดขาด
โฟลเดอร์นี้เป็นระบบแกนกลาง การแก้ไขอาจทำให้ **State Machine** ทุกตัวในโปรเจกต์เสียพร้อมกัน

สิ่งที่ทำได้อย่างปลอดภัย:

- สร้างคลาส Action/Condition ใหม่ไว้นอกโฟลเดอร์ `StateMachine` เช่นโฟลเดอร์ของ Character หรือ Feature ที่รับผิดชอบ
- สร้างและตั้งค่าไฟล์ `StateSO` , Action, Condition และ `TransitionTableSO` ผ่านเมนู Create และ Inspector
- นำ `StateMachineController` ไปใส่ GameObject และกำหนด Transition Table

2. ภาพรวมการทำงาน

Input / Gameplay Data → Condition → Transition Table → State → Action

ส่วนประกอบ	หน้าที่	ตัวอย่าง Rin
StateMachineController	สร้างและอัปเดต State Machine จาก Transition Table ที่กำหนด	Component บน GameObject <code>Rin StateMachine</code>
TransitionTableSO	กำหนด Initial State, Any State และ Local Transition	<code>PlayerLocomotionTransitionTable</code>
StateSO	รวม Action ที่ต้องทำขณะอยู่ใน State	<code>MoveState</code> , <code>RunState</code> , <code>IdleState</code>
StateActionSO	สร้างพฤติกรรม Runtime เช่นปรับ animation multiplier	<code>PlayerWalkAnimationAction</code>
StateConditionSO	ตรวจสอบเงื่อนไขและคืนค่า true/false	<code>IsRunningInputCondition</code>

หนึ่ง **Controller** ใช้หลาย **Transition Table** ได้ แต่ละ Table จะกลายเป็น State Machine แยกชุดและทำงานพร้อมกัน เหมาะกับการแยก Locomotion กับ Upper Action

3. ศึกษาตัวอย่าง Rin

- เปิด Scene `Assets/YukiLearningStateMachine/Scene/TestStateMachine.unity`
- เลือก GameObject ชื่อ `Rin StateMachine` ใน Hierarchy
- ดู Component `StateMachineController` ของ **Transition Tables**

4. เปิด asset `PlayerLocomotionTransitionTable` เพื่อดู Initial State และ Transition

5. เปิด State และ Action/Condition ที่ถูกอ้างอิงต่อเนื่อง เพื่อเห็นการประกอบระบบครบสาย

โครงสร้าง Locomotion ในตัวอย่าง:

ประเภท	จาก	เงื่อนไข	ไป
Initial	เริ่มระบบ	—	<code>MoveState</code>
Any State	State ใดก็ได้	<code>Alpha1KeyCondition = true</code>	<code>IdleState</code>
Local	<code>MoveState</code>	<code>IsRunningInputCondition = true</code>	<code>RunState</code>
Local	<code>RunState</code>	<code>IsRunningInputCondition = false</code>	<code>MoveState</code>

ตัวอย่างยังมี `PlayerUpperActionTransitionTable` สำหรับสลับ `UpperIdleState` ↔ `BookState` ด้วย `IsBookInputCondition` แสดงวิธีแยก State Machine คนละ layer ของพฤติกรรม

4. วิธีสร้าง State Machine ชุดใหม่

ขั้นที่ 1 — สร้าง Action

Action คือสิ่งที่ State ต้องทำ เช่น ตั้งค่า Animator, ปรับความเร็ว, เล่นเสียง หรือเรียก logic ของ Component ให้สร้างคลาสใหม่ไว้นอกโฟลเดอร์ StateMachine โดยสืบทอดจาก `StateActionSO` และสร้าง Runtime Action ที่สืบทอดจาก `StateAction`

- `Awake(StateMachine)` : หา Component/เตรียม reference
- `OnStateEnter()` : ทำครั้งเดียวเมื่อเข้า State
- `OnUpdate()` : ทำทุกเฟรมขณะอยู่ใน State
- ใช้ callback อื่นตามที่ base class รองรับเมื่อต้องการ FixedUpdate, Exit หรือ Dispose

อ้างอิงรูปแบบจาก `PlayerLocomotionAnimationActionSO.cs` : SO เก็บค่าจาก Inspector และ Runtime Action เรียก `PlayerLocomotion.SetMuitply(...)` เมื่อเข้า State

ขั้นที่ 2 — สร้าง Condition

Condition ต้องตอบว่าเงื่อนไข “ผ่านหรือไม่ผ่าน” ให้สร้างคลาส SO ที่สืบทอดจาก `StateConditionSO` และ Runtime Condition ที่สืบทอดจาก `Condition`

- `CreateCondition()` : สร้าง instance สำหรับ Runtime
- `Statement()` : คืนค่า true/false
- หาก subscribe event ต้อง unsubscribe ใน `Dispose()`
- ตรวจสอบ null และแจ้ง Error/Warning ที่อธิบายสาเหตุได้ชัดเจน

อ้างอิงจาก `IsRunningInputConditionSO.cs` : Condition ฟัง event การวิ่งจาก Input Observer เก็บค่าไว้ และคืนค่าผ่าน `Statement()`

ขั้นที่ 3 — สร้าง Asset ของ Action และ Condition

1. ใน Project window คลิกขวาที่โฟลเดอร์ ScriptableObject ของ feature
2. เลือก **Create** → **YUKI Learning State Machine** → **Actions/...** หรือ **Conditions/...**
3. ตั้งชื่อให้สื่อความหมายและกำหนด reference/parameter ใน Inspector ให้ครบ

ขั้นที่ 4 — สร้าง State

1. เลือก **Create** → **YUKI Learning State Machine** → **State**
2. ตั้งชื่อ เช่น `WalkState`, `AttackState` หรือ `StunnedState`
3. เพิ่ม Action assets ในรายการ **Actions** ตามลำดับที่ต้องการ

State หนึ่งตัวใส่หลาย Action ได้ เช่น ตั้ง animation พร้อมเปิดเสียงและปรับ movement แต่ควรให้ Action แต่ละตัวมีหน้าที่ชัดเจน

ขั้นที่ 5 — สร้าง Transition Table

1. เลือก **Create** → **YUKI Learning State Machine** → **Transition Table**
2. กำหนด **Initial State** ซึ่งเป็น State แรกตอน Controller เริ่มทำงาน
3. เพิ่ม **Any State Transition** สำหรับเงื่อนไขที่ต้องตรวจสอบจากทุก State

4. เพิ่ม **Local Transition** แล้วกำหนด From, To และ Condition Group

5. วิธีอ่าน **AND / OR / Expected**

Inspector	ความหมาย
หลาย Condition ในกลุ่มเดียว	เชื่อมแบบ AND — ทุกข้อในกลุ่มต้องผ่าน
หลาย Condition Group	เชื่อมแบบ OR — ผ่านกลุ่มใดกลุ่มหนึ่งก็เปลี่ยน State ได้
Expected เปิด	Condition ต้องคืนค่า true
Expected ปิด	Condition ต้องคืนค่า false

ตัวอย่าง: เปลี่ยนไป Run เมื่อ มี input เคลื่อนที่ AND กดรีง หรือ เปิดโหมด Auto Run ให้สร้าง 2 กลุ่ม: กลุ่มแรกมี 2 Conditions และกลุ่มที่สองมี Auto Run Condition หนึ่งข้อ

Transition ที่ไม่มี Condition Group หรือมีกลุ่ม AND วางจะไม่ทำงาน ต้องเพิ่ม Condition อย่างน้อยหนึ่งตัวในกลุ่ม

6. ลำดับความสำคัญของ Transition

1. ระบบตรวจ **Any State Transitions** ก่อน **Local Transitions**
2. ภายในแต่ละรายการ ระบบตรวจจากบนลงล่าง
3. Transition แรกที่ผ่านจะถูกเลือกเพียงรายการเดียวในรอบนั้น

ใช้ปุ่ม **Up** และ **Down** ใน Inspector เพื่อจัด Priority ครอบงำ Transition อื่นๆ เช่น Death, Stun หรือ Cutscene ไว้เหนือ Transition ทั่วไปเมื่อการออกแบบต้องการให้มีสิทธิ์สูงกว่า

7. นำไปใช้กับ GameObject

1. เลือก GameObject ที่เป็นเจ้าของพฤติกรรม
2. Add Component `StateMachineController`
3. กำหนดขนาดรายการ **Transition Tables**
4. ลาก Transition Table ที่สร้างไว้ใส่แต่ละช่อง
5. ตรวจสอบว่า Component ที่ Action ต้องใช้อยู่บน GameObject เดียวกัน เพราะ Action ตัวอย่างเรียก `stateMachine.GetComponent<T>()`
6. เข้า Play Mode และทดสอบทุกเส้นทางเข้า/ออกของ State

หากต้องการ Locomotion และ Upper Body ทำงานคู่กัน ให้ใส่ Transition Table สองชุดใน Controller เดียวกันตามแนวคิดของ Rin แต่ตรวจสอบว่าทั้งสองชุดไม่ส่ง property เดียวกันจนขัดแย้ง

8. Checklist ก่อนส่งงาน

- ☐ ไม่ได้แก้ไฟล์ใด ๆ ใน `Script/StateMachine`
- ☐ Initial State ถูกกำหนดและไม่เป็น None
- ☐ From State, To State และ Condition ของทุก Transition ไม่เป็น None
- ☐ ทุก Condition Group มี Condition อย่างน้อยหนึ่งตัว
- ☐ Priority ของ Any State และ Local Transition เรียงถูกต้อง
- ☐ Action/Condition มี reference ใน Inspector ครบและตรวจสอบ null แล้ว
- ☐ Event ที่ subscribe ถูก unsubscribe ใน Dispose
- ☐ GameObject มี Component ที่ Action เรียกใช้
- ☐ ทดสอบ Play Mode ครบทุก State และไม่พบ Error ใน Console

9. ปัญหาที่พบบ่อย

อาการ	ตรวจสอบ
เริ่ม Play แล้ว Error หันที	ตรวจสอบ Initial State, ช่อง Transition Table และ reference ที่ Action/Condition ต้องใช้
ไม่เปลี่ยน State	ตรวจสอบ Condition Group ว่าง, Expected true/false, ค่า input และลำดับ Priority

อาการ	ตรวจสอบ
เข้า State ผิดตัว	Any State มีสิทธิ์ก่อน Local; ตรวจสอบรายการด้านบนที่อาจผ่านตลอดเวลา
NullReference ใน Action	Component ที่เรียกผ่าน GetComponent อาจไม่ได้อยู่บน GameObject เดียวกับ Controller
Event ทำงานช้าหลังเปลี่ยน Scene	ตรวจสอบว่าถอด event listener ใน Dispose แล้ว

หากพบข้อจำกัดของระบบแกนกลาง ให้แจ้งผู้ดูแลระบบก่อน ห้ามแก้ไขโค้ดในโฟลเดอร์ **StateMachine** เพื่อแก้ไขเฉพาะหน้า

เอกสารนี้จัดทำจากโครงสร้างและ asset ที่อยู่ในโปรเจกต์ Yantra ณ วันที่ 23 กรกฎาคม 2026 โดยอ้างอิง Scene TestStateMachine และตัวอย่าง Rin